大整数素性检测与素数生成器项目 primora 介绍

PB26000223 董双瑞

2026 年 11 月 10 日

1 项目介绍

primora 是一个使用 C99 实现的单线程大整数素性检测与素数生成工具，主要使用 Miller Rabin 素数概率判断算法和 Montgomery 模乘算法。

1.1 项目使用

项目地址在https://www.github.com/GDJT-DSR/primora中，使用 CMake 构建，GCC / Clang / MSVC 编译器、Debug / Release 模式均支持，编译后生成 primora 可执行文件，该程序子命令如下：

- gen <hex_digits>：生成指定位数的素数，结果保存在 prime.txt（可通过 -o 指定）中；
- verify 0x<hex>：对给定十六进制整数做概率素性检测。

该程序还支持以下参数：

- -s：设置随机数种子，默认为 time(NULL)；
- -t：设置 Miller Rabin 算法的次数，默认为 10。

1.2 项目结构和依赖

项目文件及相应功能：

- rand：随机数生成器，使用高质量高速长周期（约 $2^{256}-1$）的伪随机数生成器 xoshiro256** 作为主要随机算法，其中初始状态使用 SplitMix64 初始化；
- bi：大整数模拟，使用无符号整数 item_t 数组存储（小端序），并实现了基础的加减乘模；
- mont：Montgomery 模乘算法部分，实现了模乘、模幂等功能；
- utils：字符串比较、长度计算等基础算法；
- consts：一些预计算常数，包含小素数表；
- test：包含 gen 和 verify 功能的主要逻辑，供 main 函数调用；
- main：负责解析参数、调用对应函数、输出最终结果等。

项目依赖：

- stdio：用于处理输入和输出；
- stdlib：用于分配、释放内存；
- time：作为默认种子。

2 项目性能

2.1 统计数据

测量环境如下：

- 机器：2 核 2GB，Debian 6.12.107 amd64；
- 编译：GCC 14.2.0；使用`-O3`选项编译；
- 任务：运行 `primora gen 1024 i`，其中 `i` 从 1 到 10000

结论：平均 9.11s，最大 83.15s，详细数据见 bench.txt，采集到的火焰图如图1所示

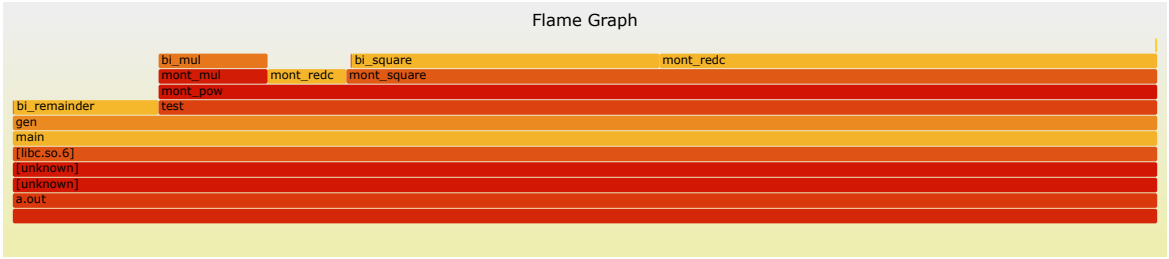


图 1: 项目使用 perf 采集到的火焰图

2.2 性能优化部分

对于判断素数时，先进行小素数试除，以减少 Miller Rabin 算法的次数，进而提升性能，本项目中使用 200000 以内的素数（共 17983 个）进行试除，这些数已经硬编码进项目的`consts.h`,`consts.c`中。

大整数表示时，尽可能使用更大位长得无符号整数，支持 32/64 位（取决于是否有 `__int128`，因此 msvc 只有 32 位）；对平方这一对称运算做了单独优化，交叉项只计算一遍省去一些计算开销。

Montgomery 算法中，把模数 N 的逆元计算缩减到只计算 N 最低位的逆元，具体计算可动态推出整个数组；在 `mont_mul` 中采用 CIOS 算法，较 SOS 实现性能更好；在 `mont_pow` 中采用滑动窗口和快速幂结合的方法，一次读取若干位（默认至多 8，通过 `WINDOW_SIZE` 宏设置这个最大值），用空间换时间，可以减少快速幂过程中乘法的次数；

大部分函数签名使用 `restrict` 修饰，可以一定程度加快数据访问。